sEMG Gesture Recognition with Mamba: Project Proposal

Oct 7, 2026 · @n

Predicting hand gestures from muscle signals, and testing whether Mamba helps

Document	Project proposal, AI Club
Duration	12 weeks
Data	Three public sEMG datasets: GRABMyo, Mendeley multi-channel sEMG, Kaggle EMG gesture recognition
Models compared	Hand-made features with KNN, SVM and Random Forest; 1D CNN; GRU; Transformer; Mamba
Version	Draft 1, 8 October 2026

Table of contents

- Our aim at a glance
- 1. Summary
- 2. The problem, explained from scratch
- 3. Goals, objectives and research questions
- 4. The datasets
- 5. Background: what exists, what is missing, why Mamba
- 6. Proposed method
- 7. Week-by-week plan
- 8. Evaluation plan and expected outcomes
- 9. Risks
- 10. Learning path and glossary

Our aim at a glance

We will predict which hand gesture a person made, using only the electrical signals from their forearm muscles.

Part	What it is
What goes in	A short slice of sEMG signal (about 200 ms): a small table of numbers, one column per electrode
What comes out	One gesture label, such as wrist flexion, wrist extension, fist or rest
Type of problem	Classification, like deciding whether a photo shows a cat or a dog

What we will find out

1. How much accuracy drops when the person is new to the model, instead of a random split of the same recordings.
2. How much accuracy drops on a recording day the model has not seen.
3. Which method is best: hand-made features with KNN or SVM, CNN, GRU, Transformer or Mamba.
4. Whether Mamba matches or beats the others, and whether it is smaller or faster.
5. Which gestures the models confuse most.

Why it matters. The paper we follow reports 97.43%, but that number probably comes from testing on data very similar to the training data. We want an honest result for people the model has never seen, because that is what a real prosthetic or controller would face.

In one sentence: we train models to predict hand gestures from muscle signals, and we find out which one stays accurate on new people and new days, including whether Mamba is better.

1. Summary

We will build a program that looks at the electrical signals from forearm muscles and tells which hand gesture the person made. We will test it on three public datasets, so the result does not depend on one lab's recordings. We compare a classical method (hand-made features plus KNN or SVM), standard deep networks (CNN, GRU, Transformer) and

Mamba, a newer sequence model. The key rule is honest testing: every model must work on **people it has never seen** and on **recordings from a different day**. The project runs 12 weeks, starts from zero knowledge, and ends with a reproducible code base, a results table and a short report.

2. The problem, explained from scratch

What is sEMG? When you move your hand, muscles in your forearm produce tiny electrical signals. Small stickers (electrodes) on the skin pick them up. This is called surface electromyography, or sEMG. It is painless and needs no surgery.

What does the data look like? Just a table of numbers. Each column is one electrode (a *channel*) and each row is one moment in time. The signal is measured thousands of times per second (the *sampling rate*). For example, one 5-second recording in the GRABMyo dataset has 10,240 rows and 32 columns.

What is gesture recognition? The task is to take a short slice of that table (a *window*, for example 200 milliseconds) and output a label such as "wrist flexion" or "fist". This is a **classification** problem, the same kind as deciding whether a photo shows a cat or a dog.

Why does it matter? A reliable decoder can control a prosthetic hand, a wheelchair, a game or a computer without a keyboard or a camera.

Why is it hard?

1. **Every person is different.** Muscle size, skin and electrode position change the signal, so a model trained on some people often fails on a new person.
2. **Every day is different.** Sweat, fatigue and small shifts of the electrodes change the signal even for the same person.
3. **The signal is noisy.** Power-line hum, movement and electrode contact add noise that has nothing to do with the gesture.

Why testing is the real problem. Many papers cut one set of recordings into random windows and report 95% or more. Neighbouring windows from the same recording look almost identical, so the model is partly tested on data it has already seen. When you test on new people, accuracy usually drops sharply. Our project measures this drop and tries to shrink it.

Problem statement in one line: Can we classify hand gestures from sEMG windows accurately for new people and new days, and does a Mamba model do this better than classical methods and standard deep networks?

3. Goals, objectives and research questions

Main goal. Build and fairly compare sEMG gesture classifiers, including Mamba, that stay accurate on new people and new days.

Objectives. Each objective is something we can check at the end.

#	Objective	How we know it is done
O1	Understand sEMG, windows, features and the three datasets	Every member passes a short quiz and writes a dataset note
O2	Build one preprocessing pipeline for all datasets	One command turns raw files into `data.h5` plus split files, with passing leakage tests
O3	Build a classical baseline	Results table for KNN, SVM and Random Forest on both split types
O4	Build deep models: CNN, GRU, Transformer	Trained on every dataset with 5 seeds each
O5	Build a Mamba model and compare it fairly	Same data, same splits, same training script as the other models
O6	Evaluate honestly and explain the results	Mean and standard deviation, confusion matrices, ablations, error analysis
O7	Document everything	README, report, and code that anyone can re-run

Research questions

1. **RQ1.** How much does accuracy fall when we test on new people instead of a random split of windows?
2. **RQ2.** Does a deep model beat hand-made features plus KNN or SVM when tested on new people?
3. **RQ3.** Does Mamba match or beat a Transformer and a CNN, at a similar or lower number of parameters and speed?
4. **RQ4.** How much does accuracy fall on a different recording day, and which gestures get confused most?

Out of scope. Real-time hardware, building our own electrode device, and recording new data. These can be future work.

4. The datasets

We use three public datasets that differ in size, electrodes and structure. Together they show whether a method works beyond one lab. The facts below come from the dataset pages on 8 October 2026. Where the page did not say, we write "not stated" and the person working on that dataset must find it in the files or the paper.

	GRABMyo	Mendeley multi-channel sEMG	Kaggle EMG gesture recognition
Source	PhysioNet, v1.1.0	Mendeley Data, v2	Kaggle
Participants	43 healthy adults	40	Not stated on the page we could read
Recording days (sessions)	3 days, same protocol each day	Not described	Not stated
Gestures	16 hand and wrist gestures plus rest (17 labels)	10 incl. rest: wrist extension, wrist flexion, ulnar deviation, radial deviation, grip, finger abduction, finger adduction, supination, pronation	Not stated
Repetitions	7 trials per gesture, 5 s each	5 cycles of the 10 gestures per person	Not stated
Channels	16 forearm + 12 wrist (plus 4 unused, 32 recorded)	4 forearm muscles	Not stated
Sampling rate	2048 Hz	Not stated	Not stated
Format	WFDB (.dat, .hea), MATLAB conversion script provided	Not stated	Not stated

	GRABMyo	Mendeley multi-channel sEMG	Kaggle EMG gesture recognition
Licence	CC BY 4.0	Not specified	Not stated

What each dataset is good for

- **GRABMyo** is the richest. It has many people and three separate days, so it supports **both** the new-person test and the new-day test. Its files are named `session<i>_subject<j>_gesture<k>_trial<t>`, which gives us the labels for free. It is large (about 9.4 GB), so plan disk space and download time.
- **Mendeley** has 40 people and only 4 channels, so it tests how well a method works with very little sensor information. It supports a new-person test. Because it does not describe separate days, we cannot assume a new-day test.
- **Kaggle** is the one we know least about. Before anyone plans around it, we must open its files, count subjects and classes, and record the sampling rate.

Why the differences matter. A 2048 Hz dataset can be filtered up to a few hundred Hz, while a slower one cannot. Sampling rate and channel count decide filter settings and the input shape of every model, so the first job for each dataset is to fill in the "not stated" cells.

Reference dataset from the paper. The paper we follow (Miah et al., arXiv 2408.13723) uses an 8-channel, 36-subject armband dataset from the UCI repository. We do not train on it, but we reproduce its style of result for comparison.

5. Background: what exists, what is missing, why Mamba

The reference paper. Miah et al. (arXiv 2408.13723) compute 23 hand-made features per signal, keep the best ones with an Extra Trees selector and classify with machine-learning models. Their best result is 97.43% with KNN on a 36-subject armband dataset. It is a clear, simple recipe and a good starting point for us.

What the paper does not show. It does not report a test on people who were absent from training, and it uses no deep sequence model. That is the gap we fill. The high number is most likely a random-window result, so we reproduce it and then also measure the new-person result.

Three ways to read a signal window

Family	Idea in plain words	Models we use
Hand-made features	A person decides which summary numbers matter (average strength, number of zero crossings, and so on), then a simple classifier uses them	KNN, SVM, Random Forest
Learn local patterns	The model slides small filters along time to find short shapes	1D CNN
Learn over the whole sequence	The model reads the full window and decides what to remember or attend to	GRU, Transformer, **Mamba**

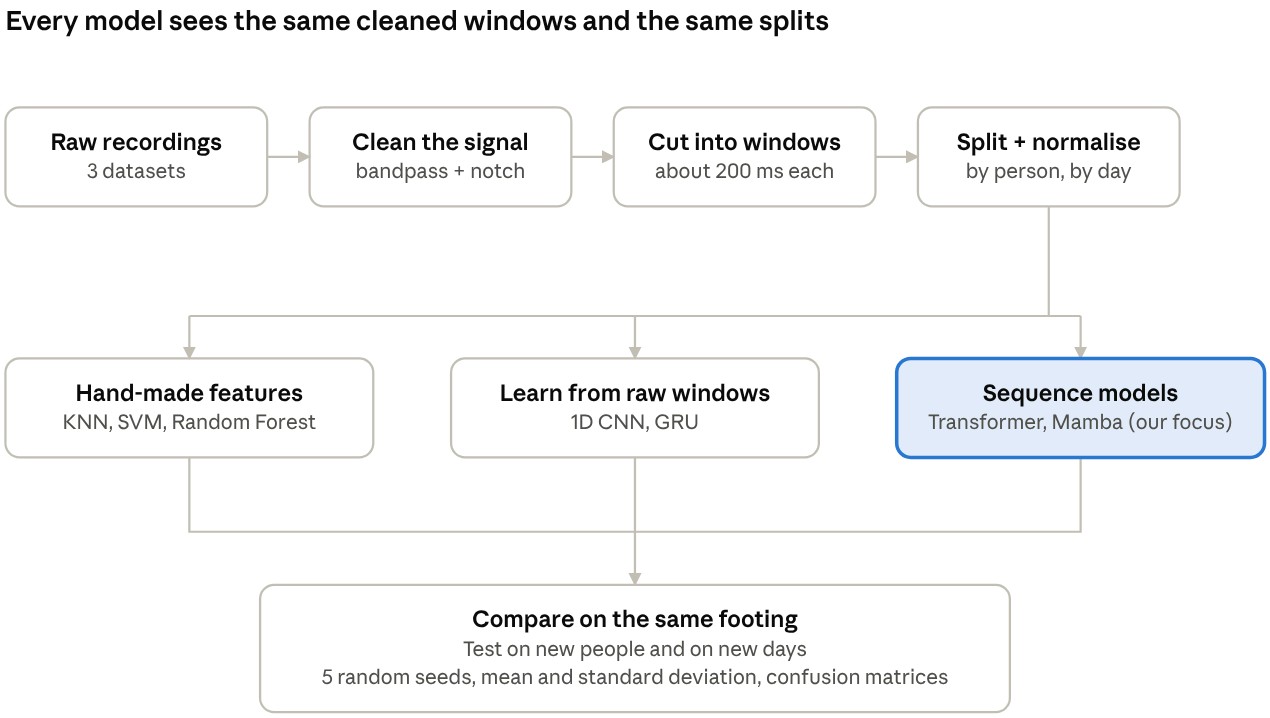
What is Mamba? Mamba is a sequence model from Gu and Dao (December 2023, arXiv 2312.00752; described here from memory, to be re-checked by whoever implements it). Think of a person reading a sentence while keeping a short memory. A classic recurrent network updates that memory in a fixed way. Mamba lets the **current input decide** what to keep and what to forget, which is why it is called a *selective state space model*. Its cost grows in step with the sequence length, while a Transformer's attention cost grows with the **square** of the length.

Why test it here. sEMG is a time series, and a gesture decoder may one day run on a small, low-power device. A model that is accurate and cheap is attractive.

An honest expectation. Our windows are short (about 200 ms), so Mamba's speed advantage may be small and it may not beat a well-tuned CNN. We treat that as a possible result, not a failure. The point is to measure it properly: same data, same splits, same training script, several random seeds.

6. Proposed method

One shared pipeline cleans the data and builds the splits, then three families of models are trained on exactly the same inputs.



method pipeline · 4 prep steps, 3 model families, 1 comparison

Read it left to right, then down. The first row prepares the data once. The second row is where models differ. The last box is where we compare them, and it is the same test for every model.

Settings fixed for every model

- Window of about 200 ms with a 100 ms step, and a 100 and 300 ms variant for an ablation.
- Normalisation numbers are computed on training data only.
- AdamW optimiser with learning rate 1e-3 and cosine decay, early stopping on validation macro-F1.
- Seeds 0 to 4 for every run, and models of roughly 1 million parameters or fewer.

Mamba model recipe

1. Project the channels of each time step to a vector of size 64.
2. Stack 2 to 4 Mamba blocks, each with a residual connection and layer normalisation.
3. Average over time, then a linear layer gives one score per gesture.
4. Use the `mamba-ssm` package on a GPU, or a pure PyTorch version if no GPU is available.
5. Match the number of parameters of the Transformer so the comparison is fair, and record training and inference time.

7. Week-by-week plan

The project takes 12 weeks and is built so that someone starting from zero can keep up. Learn the idea first, then build it. Three gates (end of weeks 4, 6 and 9) must be passed before the next stage starts.

Week	Goal	Main tasks	Deliverable
1	Understand the problem and set up	Read sections 2 and 5 of this proposal; install Python, NumPy, SciPy, scikit-learn, PyTorch and h5py; learn basic Git; create the shared repo; download the datasets	Working environment for everyone, repo with README, short quiz on sEMG, windows and splits
2	Know the data	Open every file; plot raw signals per channel and per gesture; count subjects, sessions, classes; fill every "not stated" cell in section 4	One note per dataset (`docs/dataset_<name>.md`) and an EDA notebook
3	Clean the signal and cut windows	Choose bandpass and notch settings from each dataset's sampling rate; resample to one common rate; cut windows of about 200 ms; drop windows that mix two labels	Windowed arrays for every dataset
4	Leak-free splits (**Gate 1**)	Compute normalisation on training data only; build the new-person split and, where valid, the new-day split; write automated leakage tests	`data.h5` and split files per dataset; all tests pass

Week	Goal	Main tasks	Deliverable
5	Classical baseline	Compute features (MAV, RMS, WL, ZC, SSC, PTP); train KNN, SVM, Random Forest; run both a random split and the new-person split	Baseline table; first answer to RQ1
6	Shared training code (**Gate 2**)	Add Extra Trees feature selection; write one training script driven by config files; train the 1D CNN	Baseline table agreed by the team; working CNN
7	GRU and Transformer	Watch the Transformer video; implement and train GRU and Transformer on one dataset first, then all	Results for both models
8	Mamba	Watch the Mamba and SSM video; implement the Mamba model from section 6; match its parameter count to the Transformer; check that its training loss falls on one dataset	Mamba trains and passes a sanity check
9	Full runs (**Gate 3**)	Train every model on every valid dataset and split with 5 seeds; tune only on validation people	Complete raw results; every model runs everywhere

Week	Goal	Main tasks	Deliverable
10	Evaluate and explain	Ablations (window length, filtering, augmentation); confusion matrices; best and worst people; parameters and speed per model	Final results table and plots; answers to RQ1 to RQ4
11	Write up	Write the report using the results; clean the code; finish the README	Draft report, final figures, tidy repo
12	Buffer and presentation	Someone who did not write the code runs everything from a fresh clone; fix gaps; prepare slides	Tagged final repo and presentation

If we fall behind. Cut in this order: the cross-dataset ablation, then GRU, then extra ablations. Never cut the leakage tests, the baseline or the Mamba comparison, because they are the point of the project.

Weekly rhythm. One short team meeting per week. Each person says what they finished, what blocks them and what they do next. Blocked for more than a day means ask early.

8. Evaluation plan and expected outcomes

The rule. Tune settings on validation people. Report on test people once. Any change made after seeing test results makes the test meaningless.

Two tests

- **New-person test.** Whole people are assigned to train, validation or test. Nobody appears in two sets.
- **New-day test.** Train on earlier days of each person and test on a later day. This only applies to a dataset with several recording days, which from the pages we read means GRABMyo.

Measures

Measure	Why we use it
Macro F1 (main)	Averages the score over gestures so a rare gesture counts as much as a common one
Accuracy	Familiar headline number; can mislead when gestures are unevenly represented
Confusion matrix	Shows which gestures get mixed up, for example wrist flexion and extension
Parameters and time per window	Shows whether a model is small and fast enough for a real device
Mean and standard deviation over 5 seeds	Shows whether a gap between models is real or luck

Expected outcomes. These are hypotheses to test, not promises.

1. Accuracy on new people will be clearly lower than on a random window split (RQ1).
2. A deep model will help on new people only if it is trained with augmentation and kept small (RQ2).
3. Mamba will probably be close to a Transformer and a CNN on short windows. Its advantage, if any, will show in speed and size more than in accuracy (RQ3).
4. Accuracy on a new day will be lower than on a new window of the same day, and confusions will concentrate on similar wrist gestures (RQ4).

What we will hand in. A results table (model by dataset by test type), confusion matrices, ablation plots, a short written explanation, and code that regenerates every number.

9. Risks

Risks

Risk	What to do
A dataset has missing details (sampling rate, subjects, classes)	Find them in the files and paper in week 2; if impossible, drop that dataset's session test and say so

Risk	What to do
Datasets are large (GRABMyo is about 9.4 GB)	Check disk space and download early; process one subject at a time
Data leaks between train and test	Automated leakage tests from week 4; no model work before they pass
Mamba is hard to install or needs a GPU	Use Colab or Kaggle GPUs, or a pure PyTorch version; decide in week 7
Models overfit small data	Small models, augmentation, early stopping, 5 seeds
Someone falls behind	Short weekly check-in; pair up; use the cut order in section 7
Results are worse than the paper's number	Expected for new-person tests; explain the gap as a finding

10. Learning path and glossary

What to learn, and when. Learn only what the current week needs.

Weeks	Learn	How
1	Python basics, NumPy arrays, Git	Your Git cheatsheet; practise on the repo
2 to 3	Reading signals, plotting, filtering, sampling rate	Plot a signal before and after a filter and compare
4	Why train, validation and test sets must not overlap	Explain leakage in your own words to a teammate
5 to 6	Classification with scikit-learn, features, KNN and SVM	Reproduce a small example, then apply it to windows
6 to 7	Neural networks in PyTorch, CNN, GRU, attention	PyTorch beginner tutorials; Transformers explained
8	State space models and Mamba	Mamba and SSM explained, then the Mamba paper

Glossary

Term	Plain meaning
sEMG	Muscle electrical activity measured with stickers on the skin
Channel	One electrode's signal, one column of the data
Sampling rate	Measurements per second, in Hz
Nyquist limit	Half the sampling rate; the highest frequency the data can hold
Bandpass / notch filter	Keep a useful frequency range / remove one noisy frequency such as 50 or 60 Hz mains hum
Window	A short slice of signal (for example 200 ms) that becomes one example
Feature	A summary number computed from a window
Subject-wise split	Test on people the model never saw
Session-wise split	Test on a recording day the model never saw
Leakage	Test information sneaking into training, which inflates scores
Macro F1	A score averaged over gestures so each counts equally
Seed	The starting number for randomness; several seeds show how stable a result is
Mamba	A sequence model with an input-dependent memory (selective state space model)

Sources and what is unverified

- GRABMyo, PhysioNet v1.1.0 and Mendeley dataset v2: facts in section 4 are from these pages as read on 8 October 2026.
- Kaggle EMG gesture recognition: the page text we could read gave only the title, so every detail must be checked from the files.
- Miah et al., arXiv 2408.13723: read from the PDF in the club folder.

- Mamba (Gu and Dao, arXiv 2312.00752): described from memory and not re-read for this proposal.
- The Mendeley page does not state a sampling rate, file format or licence, so they must be found in the files.